



Universidade do Minho
Licenciatura em Engenharia Informática

Unidade Curricular de Laboratórios de Informática III

Relatório Fase I

Ano Lectivo de 2025/2026

Gonçalo Fernandes da Silva e Sousa
A110977 - goncalofssousa

Pedro Tiago Moniz Moraes
A109719 - tiagomoraes121

Tiago da Cunha Pereira
A112010 - tiagocpereira13

Índice

1. Introdução	3
2. Sistema	4
2.1. Arquitetura definida	5
3. Discussão	6
4. Conclusão	7

Introdução

O trabalho prático teve como objetivo o desenvolvimento de um programa capaz de processar, validar e consultar conjuntos de ficheiros fornecidos em formato CSV. Este projeto insere-se num contexto de verificação de informação, onde é fundamental garantir a correção dos dados de entrada, eficiência e tempo de execução do programa.

O sistema foi concebido para ler os diferentes datasets disponibilizados e realizar uma validação rigorosa dos mesmos. Os resultados obtidos foram posteriormente comparados com os ficheiros de saída esperados, permitindo testar e verificar o comportamento do programa de forma automática e controlada. Ainda assim, tivemos de criar e executar um conjunto de queries e posteriormente efetuar um programa-testes para a verificação das mesmas.

Entre as principais características do projeto destaca-se a implementação de hashtables, que desempenhou um papel essencial na organização e gestão eficiente dos dados. Esta estrutura permitiu associar de forma direta e rápida cada elemento às respetivas *keys*, facilitando o acesso e a manipulação da informação ao longo do programa. A biblioteca GLib revelou-se fundamental, ao fornecer um conjunto de estruturas de dados e funções utilitárias que contribuíram para um desenvolvimento mais eficiente e seguro. Por fim, a organização do código foi também uma prioridade, sendo os diferentes componentes distribuídos por múltiplos ficheiros `.c` e `.h`. Os ficheiros `.h` foram utilizados para declarar as estruturas de dados e as funções, permitindo separar as definições das funções da sua implementação real, garantindo assim um código mais estruturado e de fácil manutenção.

Sistema

O sistema foi concebido de forma modular, dividindo-se em diferentes componentes, cada um responsável por uma área específica de funcionalidade. Esta abordagem facilita a organização do código, a manutenção e a expansão do programa, permitindo ainda uma leitura mais clara das responsabilidades de cada módulo.

A estrutura central do programa assenta num conjunto de módulos dedicados às diferentes entidades do sistema como *aircrafts*, *airports*, *flights*, *passengers* e *reservations*. Cada entidade é implementada num ficheiro *.c*, onde se encontra toda a lógica associada ao seu processamento, enquanto o respetivo ficheiro *.h* contém as definições das structs e as declarações de funções necessárias. Esta separação clara entre definição e implementação permitiu um desenvolvimento mais organizado, modular e facilmente escalável.

Para além destes módulos, o sistema integra um componente dedicado à leitura de ficheiros, o *files.c*, responsável por processar os datasets em formato CSV. Este módulo estabelece a ligação entre o Input e a fase de processamento, encaminhando cada linha para as funções apropriadas e garantindo que os dados são validados e inseridos corretamente nas estruturas de dados.

A arquitetura inclui ainda módulos transversais, como o de inicialização, que prepara todas as estruturas necessárias (incluindo hashtables e listas), e o Interpreter, que recebe as queries e direciona a execução para a função correspondente. Finalmente, a componente de Output trata da criação dos ficheiros de resultados e dos registos de erros, completando o fluxo (Input → Processamento → Output) representado na arquitetura.

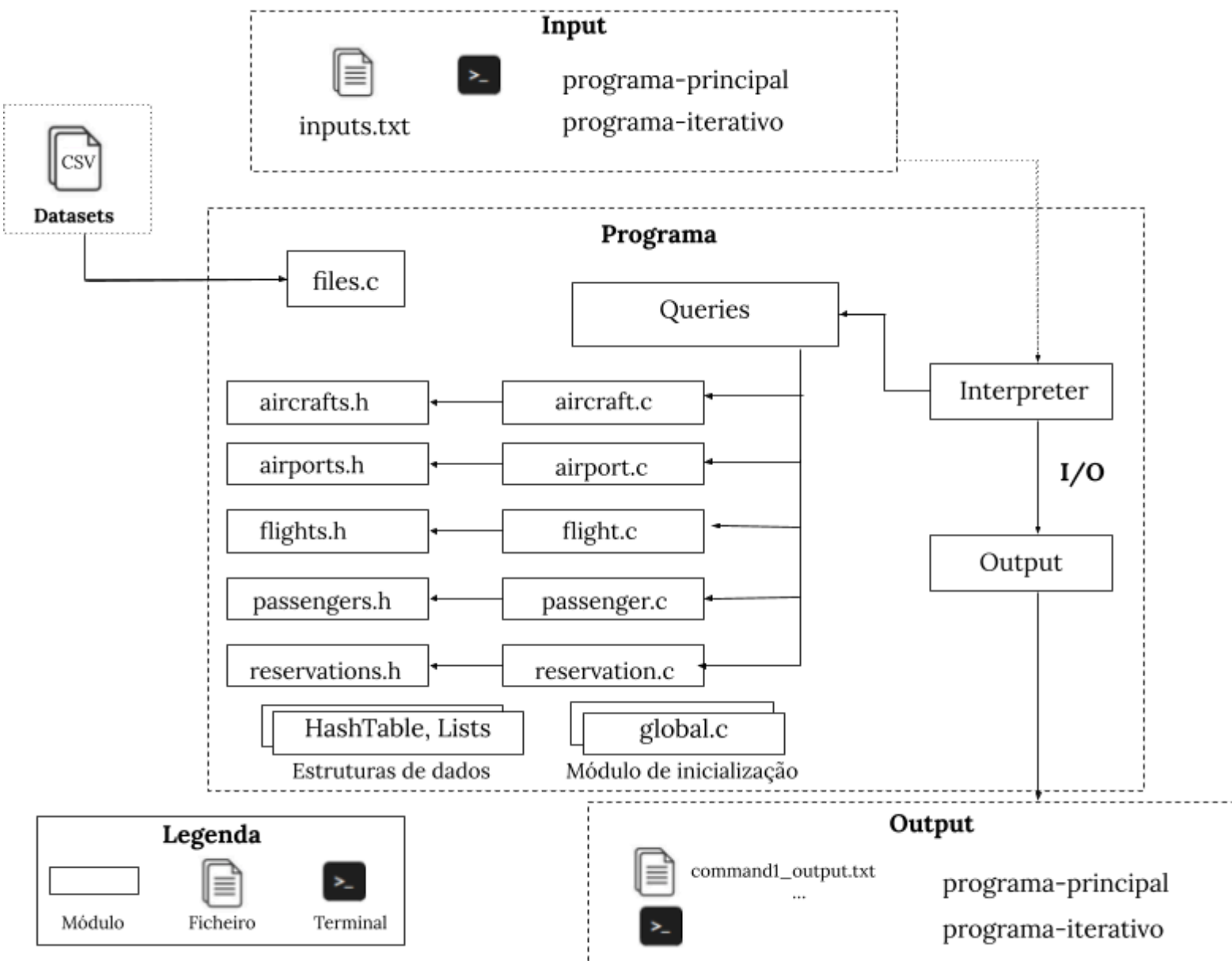


Figura 1: Arquitetura para a aplicação desenvolvida.

Discussão

No desenvolvimento do programa, foram aplicadas técnicas de encapsulamento para controlar o acesso aos dados internos dos módulos. As estruturas opacas foram utilizadas para esconder a implementação interna das estruturas de dados, permitindo que apenas as funções disponibilizadas sejam usadas para interagir com elas. Para aceder a propriedades específicas sem expor diretamente os dados, foram implementadas funções getters, que retornam valores de forma segura. Esta abordagem garante que qualquer manipulação ou leitura de dados siga regras definidas pelo módulo, evitando alterações indevidas. Além disso, a partilha de informação entre módulos foi cuidadosamente controlada: os dados nunca são expostos diretamente, sendo sempre acedidos através das funções, mantendo a integridade e consistência do programa.

Para suportar a manipulação eficiente dos dados, foram utilizadas tabelas de hash. Esta estrutura foi escolhida devido à sua capacidade de realizar operações de pesquisa, inserção e acesso em tempo praticamente constante, mesmo quando o volume de dados é elevado, como neste caso. Assim, tornou-se possível identificar rapidamente elementos com base nas respetivas chaves, facilitando a procura dos dados juntamente com a eficiência.

Os dados das aircrafts foram armazenados em hashtables, garantindo acesso rápido e eficiente a cada aircraft através da sua chave única. Na query2, foi utilizada uma lista ligada (GList) para percorrer os aircrafts numa ordem específica, permitindo processar os elementos de forma sequencial e respeitando a ordem definida, sem necessidade de alterar a estrutura principal da hashtable.

O programa-testes foi desenvolvido para validar automaticamente os resultados das queries implementadas, comparando os ficheiros de saída gerados pelo programa com os outputs esperados. Para cada query, o programa verifica se todas as linhas coincidem, identifica a primeira linha onde ocorre uma discrepância e regista o número total de testes realizados, assim como os testes que passaram com sucesso. Adicionalmente, o programa mede o tempo de execução de cada query e o tempo total, permitindo uma análise de desempenho detalhada.

Nos testes realizados com o **dataset sem erros**, todas as queries obtiveram 100% de sucesso, em todas as máquinas:

Q1: 40 de 40 testes; **Q2:** 40 de 40 testes; **Q3:** 40 de 40 testes;

Memória Utilizada: 606 MB

Nos testes realizados com o **dataset com erros**, todas as queries obtiveram 100% de sucesso, em todas as máquinas:

Q1: 40 de 40 testes; **Q2:** 40 de 40 testes; **Q3:** 40 de 40 testes;

Memória Utilizada: 610 MB

Máquina	Datasets sem erros	Dataset com erros
Linux	6s	8s
MacBook	7s	9s

Figura 2: Tabela com o tempo total nas 3 máquinas usadas (2 Linux e 1 MacBook).

Os resultados obtidos pelo programa-testes mostram um desempenho consistente e confiável em todos os datasets utilizados. No dataset sem erros, todas as queries apresentaram 100% de sucesso, o que indica que a implementação das funções e das estruturas de dados está correta para entradas válidas. Este resultado demonstra que a lógica de processamento, a utilização das hashtables e a ordenação das listas ligadas funcionam como esperado, permitindo acesso rápido aos dados e geração correta dos outputs.

No dataset com erros, os resultados também foram consistentes, apesar da presença de entradas inválidas. O programa conseguiu identificar corretamente as linhas problemáticas, registá-las quando necessário e continuar o processamento das restantes entradas. O facto de todos os testes terem produzido os outputs esperados confirma que as técnicas de validação e tratamento de erros foram implementadas de forma eficaz.

Em termos de desempenho, verificou-se que o dataset sem erros foi processado em cerca de 6 segundos, enquanto o dataset com erros demorou aproximadamente 8 segundos. Esta diferença está diretamente relacionada com o tempo adicional gasto na verificação de erros e no registo das linhas inválidas, o que é esperado em qualquer sistema que realiza validação de dados.

Relativamente à memória utilizada, o dataset sem erros consumiu cerca de 606 MB, enquanto o dataset com erros consumiu ligeiramente mais, cerca de 610 MB. Esta diferença pequena reflete o processamento adicional necessário para lidar com entradas inválidas, incluindo a alocação de memória para registar linhas problemáticas e eventuais duplicações de strings durante a verificação. A variação mínima indica que o programa é eficiente na gestão de memória, mesmo quando processa datasets com erros.

Conclusão

O trabalho desenvolvido foi bem sucedido e refletiu o empenho de todo o grupo. Tivemos de trabalhar com hashtables pela primeira vez, o que constituiu um desafio inicial, mas a experiência correu muito bem e conseguimos aplicar esta estrutura de forma eficaz conseguindo assim aplicar e aprender esta nova estrutura de dados para nós.

Além disso, aprendemos a aplicar encapsulamento, garantindo que os dados internos das estruturas fossem protegidos e manipulados apenas através de funções controladas. Também aprimoramos a manipulação de strings, lidando de forma segura com duplicações e validações de dados.

No geral, o grupo considera que a fase 1 do projeto foi uma experiência muito positiva, permitindo não só consolidar conhecimentos já adquiridos, mas também explorar novas técnicas de forma prática e bem sucedida.